Mathematical Prerequisites for Machine Learning

Probability & Statistics · Linear Algebra · Optimization

Scope. Self-contained notes for Supervised Learning, Unsupervised Learning, Neural Networks & Optimization, NLP, and CV.

Contents

1 Part I — Probability & Statistics **2**

1.1 The foundational objects: experiments, sample spaces, events . . . 2
1.2 Conditioning, independence, Bayes . . . 3
1.3 Random variables, PMF/PDF/CDF, joint and conditional . . . 3
1.4 Expectation: the operator behind ML losses . . . 4
1.5 Useful expectation identities . . . 5
1.6 Common distributions . . . 5
1.7 Likelihood, NLL, MLE, MAP . . . 6
1.8 Information theory (entropy, cross-entropy, KL) . . . 7
1.9 Maximum likelihood models behind common ML losses . . . 7
1.10 Sampling, LLN, CLT, and a simple concentration bound . . . 8
1.11 Evaluation: splits, metrics, calibration . . . 8

2 Part II — Linear Algebra **9**

2.1 Basic objects and operations . . . 9
2.2 Inner products, norms, and geometry . . . 9
2.3 Linear combinations, span, independence, basis . . . 10
2.4 Column space, null space, rank (defined carefully) . . . 10
2.5 Orthogonality and orthogonal matrices . . . 11
2.6 Projections, least squares, pseudoinverse . . . 11
2.7 Trace, determinant, and their properties . . . 12
2.8 Eigenvalues: how trace and determinant connect . . . 13
2.9 PSD matrices and the symmetric spectral theorem . . . 13
2.10 SVD and its relationships to eigenvalues, trace, determinant, norms . . . 14
2.11 A few advanced-but-useful identities (optional) . . . 15
2.12 Conditioning (why GD can zig-zag) . . . 15
2.13 Matrix calculus . . . 16
2.13.1 Differential rules (very useful in practice) . . . 16

2.13.2 Scalar-from-vector gradients . . . 16
2.13.3 Scalar-from-matrix gradients . . . 17
2.13.4 Jacobians you use constantly (NN building blocks) . . . 17
2.13.5 Gradients for affine layers . . . 18
2.13.6 Subgradients (optional) . . . 19

3 Part III — Optimization **19**
3.1 Optimization problems in ML . . . 19
3.2 First-order methods: gradient descent and SGD . . . 20
3.3 Step sizes, smoothness, and why learning rate is critical . . . 20
3.4 Momentum and Adam . . . 21
3.5 Second-order intuition . . . 21
3.6 Convexity and constraints . . . 21
3.7 Lagrange multipliers, KKT conditions, and constrained optimization . . . 22
3.7.1 Lagrange multipliers (equality constraints) . . . 22
3.7.2 KKT conditions (inequalities + equalities; optional) . . . 23
3.8 Deep learning optimization issues and fixes . . . 24

High-frequency cheat sheets **25**

1 Part I — Probability & Statistics

1.1 The foundational objects: experiments, sample spaces, events

Definition: Random experiment

A **random experiment** is a process whose outcome cannot be predicted with certainty (e.g., a coin flip, a sensor measurement, sampling an image from a dataset).

Definition: Sample space

The **sample space** Ω is the set of all possible outcomes ω of an experiment.

Definition: Event

An **event** is a subset $A \subseteq \Omega$. We say the event occurs if the outcome $\omega \in A$.

Definition: Probability measure

A probability measure $\mathbb{P}$ assigns each event A a number $\mathbb{P}(A) \in [0, 1]$ such that:

$$\mathbb{P}(\Omega) = 1, \quad \mathbb{P}(A) \geq 0, \quad \mathbb{P}\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} \mathbb{P}(A_i) \text{ for disjoint } A_i.$$

Key idea: Basic identities

$$\mathbb{P}(A^c) = 1 - \mathbb{P}(A), \qquad \mathbb{P}(A \cup B) = \mathbb{P}(A) + \mathbb{P}(B) - \mathbb{P}(A \cap B).$$

If $A \cap B = \emptyset$, then $\mathbb{P}(A \cup B) = \mathbb{P}(A) + \mathbb{P}(B)$.

1.2 Conditioning, independence, Bayes

Definition: Conditional probability

If $\mathbb{P}(B) > 0$, define

$$\mathbb{P}(A \mid B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}.$$

Definition: Independence of events

Events A and B are **independent** if

$$\mathbb{P}(A \cap B) = \mathbb{P}(A)\mathbb{P}(B).$$

Equivalently, $\mathbb{P}(A \mid B) = \mathbb{P}(A)$ whenever $\mathbb{P}(B) > 0$.

Definition: Bayes' rule

$$\mathbb{P}(A \mid B) = \frac{\mathbb{P}(B \mid A)\mathbb{P}(A)}{\mathbb{P}(B)}.$$

Key idea: ML translation

For class label Y and observed feature X,

$$p(y \mid x) \propto p(x \mid y)p(y).$$

This is the conceptual basis of Bayesian classifiers and MAP estimation.

Example: Why priors matter

If $p(y = \text{spam})$ is tiny, even a strong "spammy" feature may not produce a high posterior. This is a common source of confusion when interpreting probabilistic models.

1.3 Random variables, PMF/PDF/CDF, joint and conditional

Definition: Random variable

A **random variable** X is a function $X : \Omega \to \mathbb{R}$.

Definition: PMF and PDF

If X is discrete, $p(x) = \mathbb{P}(X = x)$ is the **PMF** and $\sum_x p(x) = 1$. If X is continuous, $p(x)$ is a **PDF** and $\int p(x)\,dx = 1$ with $\mathbb{P}(a \le X \le b) = \int_a^b p(x)\,dx$.

Definition: CDF

The **CDF** is $F(x) = \mathbb{P}(X \le x)$. It is nondecreasing, right-continuous, and satisfies $\lim_{x\to-\infty} F(x) = 0$, $\lim_{x\to\infty} F(x) = 1$.

Definition: Joint, marginal, conditional

For two random variables (X, Y), the joint distribution is $p(x, y)$. The marginals are $p(x) = \sum_y p(x, y)$ or $p(x) = \int p(x, y)\,dy$. Conditionals are $p(x \mid y) = p(x, y)/p(y)$ (when $p(y) > 0$).

Key idea: Chain rule for probabilities

For multiple variables:

$$p(x_1, \ldots, x_n) = \prod_{i=1}^{n} p(x_i \mid x_1, \ldots, x_{i-1}).$$

This shows up in autoregressive language models.

1.4 Expectation: the operator behind ML losses

Definition: Expectation

$$\mathbb{E}[X] = \sum_x x\,p(x) \quad \text{(discrete)}, \qquad \mathbb{E}[X] = \int x\,p(x)\,dx \quad \text{(continuous)}.$$

Definition: Expectation of a function (LOTUS)

For any function g,

$$\mathbb{E}[g(X)] = \sum_x g(x)p(x) \quad \text{or} \quad \mathbb{E}[g(X)] = \int g(x)p(x)\,dx.$$

Key idea: Linearity of expectation

For any X, Y (not necessarily independent) and scalars a, b:

$$\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y].$$

Definition: Variance

$$\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2.$$

Definition: Covariance and correlation

$$\mathrm{Cov}(X,Y) = \mathbb{E}[(X-\mathbb{E}[X])(Y-\mathbb{E}[Y])] = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y].$$

Correlation is $\rho = \mathrm{Cov}(X,Y)/\sqrt{\mathrm{Var}(X)\mathrm{Var}(Y)} \in [-1,1]$.

Key idea: Why squared error is "natural"

Squared error corresponds to Gaussian noise assumptions (later: NLL for Gaussian). This is why MSE is ubiquitous in regression.

1.5 Useful expectation identities

Definition: Indicator variable

$\mathbb{1}\{A\}$ equals 1 if event A occurs and 0 otherwise.

Key idea: Indicator tricks

$$\mathbb{E}[\mathbb{1}\{A\}] = \mathbb{P}(A).$$

If X is a nonnegative integer-valued random variable:

$$\mathbb{E}[X] = \sum_{k=1}^{\infty} \mathbb{P}(X \geq k).$$

Definition: Law of total expectation

For random variables X, Y:

$$\mathbb{E}[X] = \mathbb{E}\big[\mathbb{E}[X \mid Y]\big].$$

Definition: Law of total variance

$$\mathrm{Var}(X) = \mathbb{E}[\mathrm{Var}(X \mid Y)] + \mathrm{Var}(\mathbb{E}[X \mid Y]).$$

1.6 Common distributions

Definition: Bernoulli

$X \sim \mathrm{Bernoulli}(p)$ means $X \in \{0,1\}$ with $\mathbb{P}(X=1) = p$.

Definition: Categorical

$X \sim \mathrm{Categorical}(\boldsymbol{\pi})$ means $X \in \{1,\ldots,K\}$ with $\mathbb{P}(X=k) = \pi_k$, $\sum_k \pi_k = 1$.

Definition: Gaussian (Normal)

$X \sim \mathcal{N}(\mu, \sigma^2)$ has density

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

Definition: Multivariate Gaussian

$\boldsymbol{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ has density

$$p(\boldsymbol{x}) = \frac{1}{(2\pi)^{d/2} \det(\boldsymbol{\Sigma})^{1/2}} \exp\left(-\tfrac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right).$$

Definition: Exponential and Poisson (rate λ)

Poisson(λ) models counts in an interval. Exponential(λ) models waiting times (memoryless). These appear in event modeling and some generative settings.

Key idea: Conjugate priors (optional but useful)

Beta prior + Bernoulli likelihood; Dirichlet prior + Categorical likelihood. This is a clean Bayesian way to "smooth" probabilities.

1.7 Likelihood, NLL, MLE, MAP

Definition: Likelihood

Given observed data $\mathcal{D}$ and parameter θ,

$$L(\theta) = p(\mathcal{D} \mid \theta)$$

is the likelihood function.

Definition: Log-likelihood and NLL

If $\mathcal{D} = \{x_i\}_{i=1}^n$ and the model assumes i.i.d. data,

$$\log L(\theta) = \sum_{i=1}^n \log p(x_i \mid \theta), \qquad \mathrm{NLL}(\theta) = -\sum_{i=1}^n \log p(x_i \mid \theta).$$

Definition: MLE

$$\hat{\theta}_{\mathrm{MLE}} = \arg\max_\theta p(\mathcal{D} \mid \theta) = \arg\min_\theta \mathrm{NLL}(\theta).$$

Definition: MAP

With prior $p(\theta)$,

$$\hat{\theta}_{\mathrm{MAP}} = \arg\max_\theta p(\theta \mid \mathcal{D}) = \arg\min_\theta \big(\mathrm{NLL}(\theta) - \log p(\theta)\big).$$

Example: Regularization as a prior

A Gaussian prior $\theta \sim \mathcal{N}(0, \tau^2 I)$ implies $-\log p(\theta) \propto \|\theta\|_2^2$, which becomes ℓ_2 regularization (weight decay).

1.8 Information theory (entropy, cross-entropy, KL)

Definition: Entropy

For discrete p on X:

$$H(p) = -\sum_x p(x) \log p(x) = \mathbb{E}_{X\sim p}[-\log p(X)].$$

Definition: Cross-entropy

For true p and model q:

$$H(p, q) = -\sum_x p(x) \log q(x) = \mathbb{E}_{X\sim p}[-\log q(X)].$$

Definition: KL divergence

$$\mathrm{D}_{\mathrm{KL}}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)} = \mathbb{E}_{X\sim p}[\log p(X) - \log q(X)] \geq 0.$$

Key idea: The key identity

$$H(p, q) = H(p) + \mathrm{D}_{\mathrm{KL}}(p\|q).$$

So minimizing cross-entropy is equivalent to minimizing KL (when p is fixed).

Definition: Cross-entropy loss for classification

For label $y \in \{1, \ldots, K\}$ and predicted probabilities $\boldsymbol{p} \in \mathbb{R}^K$,

$$\ell(\boldsymbol{p}, y) = -\log p_y.$$

If using one-hot $\boldsymbol{y}$: $\ell(\boldsymbol{p}, \boldsymbol{y}) = -\sum_k y_k \log p_k$.

1.9 Maximum likelihood models behind common ML losses

Key idea: Regression and classification losses are NLLs

- Squared error $\leftrightarrow$ Gaussian likelihood.
- Logistic/softmax cross-entropy $\leftrightarrow$ Bernoulli/Categorical likelihood.

This unifies "loss design" with probability modeling.

Definition: Logistic regression model (probabilistic)

Binary $y \in \{0, 1\}$ with

$$p(y = 1 \mid \boldsymbol{x}) = \sigma(\boldsymbol{w}^\top \boldsymbol{x} + b), \quad \sigma(t) = \frac{1}{1 + e^{-t}}.$$

Example: Binary cross-entropy as Bernoulli NLL

If $\hat{p} = p(y = 1 \mid \boldsymbol{x})$, then Bernoulli NLL is

$$-(y \log \hat{p} + (1 - y) \log(1 - \hat{p})).$$

1.10 Sampling, LLN, CLT, and a simple concentration bound

Definition: i.i.d. samples

$X_1, \ldots, X_n$ are i.i.d. if they are independent and share the same distribution.

Key idea: Law of Large Numbers (LLN)

If X_i are i.i.d. with $\mathbb{E}[|X|] < \infty$, then

$$\frac{1}{n} \sum_{i=1}^{n} X_i \to \mathbb{E}[X] \quad \text{as } n \to \infty.$$

Key idea: Central Limit Theorem (CLT)

Under mild conditions, $\sqrt{n}(\bar{X} - \mathbb{E}[X])$ approaches a Gaussian distribution. This supports "error bars" and approximate confidence intervals.

Definition: Hoeffding's inequality (optional)

If $X_i \in [a, b]$ i.i.d., then

$$\mathbb{P}\left(\left| \frac{1}{n} \sum_{i=1}^{n} X_i - \mathbb{E}[X] \right| \geq \epsilon \right) \leq 2 \exp\left(-\frac{2n\epsilon^2}{(b - a)^2} \right).$$

1.11 Evaluation: splits, metrics, calibration

Definition: Train/validation/test

Train fits parameters; validation tunes hyperparameters; test is final evaluation.

Definition: Common metrics

Accuracy (classification), MSE/MAE (regression), precision/recall/F1 (imbalanced classes), log-loss (cross-entropy), AUC (ranking).

Key idea: Calibration (probabilities should mean something)

A classifier is calibrated if among predictions near 0.7, about 70% are actually correct. Cross-entropy often encourages better calibration than accuracy alone.

Common pitfall: Data leakage

Compute normalization statistics on the training set only. Do not choose hyperparameters using test performance.

2 Part II — Linear Algebra

2.1 Basic objects and operations

Definition: Scalar, vector, matrix

Scalar: $a \in \mathbb{R}$. Vector: $\boldsymbol{x} \in \mathbb{R}^d$. Matrix: $\boldsymbol{A} \in \mathbb{R}^{m \times n}$.

Definition: Matrix transpose

$\boldsymbol{A}^\top$ swaps rows and columns.

Definition: Identity and diagonal matrices

Identity: $\boldsymbol{I} \in \mathbb{R}^{n \times n}$ has ones on the diagonal.
Diagonal: $\operatorname{diag}(d_1, \ldots, d_n)$ has (i, i) entry d_i and zeros elsewhere.

Definition: Matrix multiplication

If $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ and $\boldsymbol{B} \in \mathbb{R}^{n \times p}$, then $\boldsymbol{AB} \in \mathbb{R}^{m \times p}$ with $(\boldsymbol{AB})_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$.

Key idea: Shapes are part of the mathematics

In ML derivations, half of the mistakes are shape mistakes. Write shapes above variables when in doubt.

2.2 Inner products, norms, and geometry

Definition: Dot product (inner product)

For $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^d$:

$$\langle \boldsymbol{x}, \boldsymbol{y} \rangle = \boldsymbol{x} \cdot \boldsymbol{y} = \boldsymbol{x}^\top \boldsymbol{y} = \sum_{i=1}^{d} x_i y_i.$$

Definition: Vector norms

$$\|\boldsymbol{x}\|_2 = \sqrt{\sum_i x_i^2}, \quad \|\boldsymbol{x}\|_1 = \sum_i |x_i|, \quad \|\boldsymbol{x}\|_\infty = \max_i |x_i|.$$

Definition: Cosine similarity

If $\boldsymbol{x}, \boldsymbol{y} \neq 0$, then

$$\cos\theta = \frac{\boldsymbol{x}^\top \boldsymbol{y}}{\|\boldsymbol{x}\|_2 \, \|\boldsymbol{y}\|_2}.$$

Key idea: Key inequalities

Cauchy–Schwarz: $|\boldsymbol{x}^\top \boldsymbol{y}| \leq \|\boldsymbol{x}\|_2 \, \|\boldsymbol{y}\|_2$.
Triangle inequality: $\|\boldsymbol{x} + \boldsymbol{y}\|_2 \leq \|\boldsymbol{x}\|_2 + \|\boldsymbol{y}\|_2$.

2.3 Linear combinations, span, independence, basis

Definition: Linear combination

$\sum_{i=1}^k a_i \boldsymbol{v}_i$ for scalars a_i.

Definition: Span

$\operatorname{span}\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k\}$ is the set of all linear combinations of those vectors.

Definition: Linear independence

$\boldsymbol{v}_1, \ldots, \boldsymbol{v}_k$ are independent if $\sum_i a_i \boldsymbol{v}_i = \boldsymbol{0}$ implies all $a_i = 0$.

Definition: Basis and dimension

A basis spans the subspace and is independent. Its size is the dimension.

2.4 Column space, null space, rank (defined carefully)

Definition: Column space

For $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, the column space is

$$\mathcal{C}(\boldsymbol{A}) = \{\boldsymbol{A}\boldsymbol{x} : \boldsymbol{x} \in \mathbb{R}^n\} \subseteq \mathbb{R}^m.$$

Definition: Null space

$$\mathcal{N}(\boldsymbol{A}) = \{\boldsymbol{x} \in \mathbb{R}^n : \boldsymbol{A}\boldsymbol{x} = \boldsymbol{0}\}.$$

Definition: Rank

$$\operatorname{rank}(\boldsymbol{A}) = \dim(\mathcal{C}(\boldsymbol{A})) = \text{number of independent columns.}$$

It also equals the number of nonzero singular values of $\boldsymbol{A}$ (defined later).

Key idea: Rank and data

If your dataset features are redundant (strongly correlated), the feature matrix can be (nearly) low rank. This affects identifiability and conditioning.

2.5 Orthogonality and orthogonal matrices

Definition: Orthogonal vectors

$\boldsymbol{u} \perp \boldsymbol{v}$ if $\boldsymbol{u}^\top \boldsymbol{v} = 0$.

Definition: Orthonormal set

A set is orthonormal if vectors are pairwise orthogonal and each has ℓ_2 norm 1.

Definition: Orthogonal matrix

$\boldsymbol{Q} \in \mathbb{R}^{n \times n}$ is orthogonal if

$$\boldsymbol{Q}^\top \boldsymbol{Q} = \boldsymbol{I}.$$

Key idea: What orthogonal matrices do

Orthogonal matrices preserve dot products and lengths:

$$(\boldsymbol{Q}\boldsymbol{x})^\top (\boldsymbol{Q}\boldsymbol{y}) = \boldsymbol{x}^\top \boldsymbol{y}, \qquad \|\boldsymbol{Q}\boldsymbol{x}\|_2 = \|\boldsymbol{x}\|_2 .$$

They represent rotations/reflections (no stretching).

2.6 Projections, least squares, pseudoinverse

Definition: Projection (informal)

The projection of $\boldsymbol{y}$ onto a subspace is the closest point in that subspace to $\boldsymbol{y}$ in ℓ_2 distance.

Definition: Least squares

$$\min_{\boldsymbol{w} \in \mathbb{R}^d} \frac{1}{2} \|\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}\|_2^2 .$$

Key idea: Normal equations

At the optimum:

$$\boldsymbol{X}^\top(\boldsymbol{X}\boldsymbol{w}-\boldsymbol{y})=\boldsymbol{0} \quad \Leftrightarrow \quad \boldsymbol{X}^\top\boldsymbol{X}\boldsymbol{w}=\boldsymbol{X}^\top\boldsymbol{y}.$$

Definition: Pseudoinverse (Moore–Penrose)

The pseudoinverse $\boldsymbol{X}^+$ generalizes the inverse. A canonical least-squares solution is $\hat{\boldsymbol{w}}=\boldsymbol{X}^+\boldsymbol{y}$.

Recipe: Ridge regression (stabilizing least squares)

$$\min_{\boldsymbol{w}} \frac{1}{2}\|\boldsymbol{X}\boldsymbol{w}-\boldsymbol{y}\|_2^2+\frac{\lambda}{2}\|\boldsymbol{w}\|_2^2 \quad \Rightarrow \quad \hat{\boldsymbol{w}}=(\boldsymbol{X}^\top\boldsymbol{X}+\lambda\boldsymbol{I})^{-1}\boldsymbol{X}^\top\boldsymbol{y}.$$

2.7 Trace, determinant, and their properties

Definition: Trace

For $\boldsymbol{A}\in\mathbb{R}^{n\times n}$,

$$\operatorname{tr}(\boldsymbol{A})=\sum_{i=1}^{n} a_{ii}.$$

Key idea: Trace properties

Linearity: $\operatorname{tr}(\boldsymbol{A}+\boldsymbol{B})=\operatorname{tr}(\boldsymbol{A})+\operatorname{tr}(\boldsymbol{B})$.
Cyclic: $\operatorname{tr}(\boldsymbol{A}\boldsymbol{B})=\operatorname{tr}(\boldsymbol{B}\boldsymbol{A})$ (when products are defined).
In general $\operatorname{tr}(A_1A_2\cdots A_{n-1}A_n)=\operatorname{tr}(A_nA_1\cdots A_{n-2}A_{n-1})=\cdots\operatorname{tr}(A_2A_3\cdots A_nA_1)$.
Frobenius: $\operatorname{tr}(\boldsymbol{A}^\top\boldsymbol{B})=\sum_{ij}a_{ij}b_{ij}$.

Definition: Determinant and logdet

$\det(\boldsymbol{A})$ measures volume scaling for invertible $\boldsymbol{A}$. $\log\det(\boldsymbol{A})$ appears in Gaussian models and is more numerically stable.

Key idea: Determinant properties (high-frequency)

For square matrices of compatible shapes:

$$\det(\boldsymbol{A}\boldsymbol{B})=\det(\boldsymbol{A})\det(\boldsymbol{B}), \quad \det(\boldsymbol{A}^\top)=\det(\boldsymbol{A}), \quad \det(c\boldsymbol{A})=c^n\det(\boldsymbol{A}) \quad (n\times n).$$

If $\boldsymbol{A}$ is triangular, $\det(\boldsymbol{A})$ is the product of diagonal entries.

2.8 Eigenvalues: how trace and determinant connect

Definition: Eigenvalue and eigenvector

For square $\boldsymbol{A}$, $\boldsymbol{v} \neq 0$ is an eigenvector with eigenvalue λ if $\boldsymbol{A}\boldsymbol{v} = \lambda \boldsymbol{v}$.

Definition: Similarity transform (change of basis)

Matrices $\boldsymbol{A}$ and $\boldsymbol{B}$ are **similar** if $\boldsymbol{B} = \boldsymbol{P}^{-1}\boldsymbol{A}\boldsymbol{P}$ for an invertible $\boldsymbol{P}$. They represent the same linear map in different coordinates.

Key idea: Trace and determinant are invariants

If $\boldsymbol{B} = \boldsymbol{P}^{-1}\boldsymbol{A}\boldsymbol{P}$ (similarity), then

$$\operatorname{tr}(\boldsymbol{B}) = \operatorname{tr}(\boldsymbol{A}), \qquad \det(\boldsymbol{B}) = \det(\boldsymbol{A}).$$

So trace and determinant depend on the linear map, not the coordinate system.

Key idea: Trace = sum of eigenvalues; determinant = product of eigenvalues

If $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ has eigenvalues $\lambda_1, \ldots, \lambda_n$ (counted with multiplicity), then

$$\operatorname{tr}(\boldsymbol{A}) = \sum_{i=1}^{n} \lambda_i, \qquad \det(\boldsymbol{A}) = \prod_{i=1}^{n} \lambda_i.$$

If $\boldsymbol{A}$ is positive definite (all $\lambda_i > 0$), then

$$\log\det(\boldsymbol{A}) = \sum_{i=1}^{n} \log \lambda_i.$$

These identities explain why log-determinants appear with eigenvalues in Gaussian models.

Example: A concrete 2×2 check

Let $\boldsymbol{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$. Then $\operatorname{tr}(\boldsymbol{A}) = a + d$ and $\det(\boldsymbol{A}) = ad - bc$. Its eigenvalues satisfy $\lambda_1 + \lambda_2 = a + d$ and $\lambda_1\lambda_2 = ad - bc$.

2.9 PSD matrices and the symmetric spectral theorem

Definition: Symmetric matrix

$\boldsymbol{A}$ is symmetric if $\boldsymbol{A}^\top = \boldsymbol{A}$.

Definition: Positive semidefinite (PSD)

A symmetric matrix $\boldsymbol{A}$ is PSD, written $\boldsymbol{A} \succeq 0$, if

$$\boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x} \geq 0 \quad \forall \boldsymbol{x}.$$

If the inequality is strict for all $\boldsymbol{x} \neq 0$, then $\boldsymbol{A}$ is positive definite (PD), written $\boldsymbol{A} \succ 0$.

Key idea: Spectral theorem (symmetric matrices)

If $\boldsymbol{A}$ is symmetric, then it can be diagonalized with an orthogonal matrix:

$$\boldsymbol{A} = \boldsymbol{Q}\mathrm{diag}(\lambda_1, \ldots, \lambda_n)\boldsymbol{Q}^\top, \quad \boldsymbol{Q}^\top \boldsymbol{Q} = \boldsymbol{I}.$$

For PSD matrices, all eigenvalues satisfy $\lambda_i \geq 0$.

Key idea: Why PSD matters in ML

Covariance matrices are PSD. Quadratic forms with PSD matrices behave like squared distances/energies. Log-determinants of PD matrices appear in Gaussian likelihoods and in some regularizers.

2.10 SVD and its relationships to eigenvalues, trace, determinant, norms

Definition: Singular value decomposition (SVD)

For any $\boldsymbol{A} \in \mathbb{R}^{m \times n}$:

$$\boldsymbol{A} = \boldsymbol{U}\boldsymbol{S}\boldsymbol{V}^\top,$$

where columns of $\boldsymbol{U}$ and $\boldsymbol{V}$ are orthonormal and $\boldsymbol{S}$ is diagonal with singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$.

Key idea: Eigenvalues of $\boldsymbol{A}^\top \boldsymbol{A}$ and singular values

$$\boldsymbol{A}^\top \boldsymbol{A} = \boldsymbol{V}\boldsymbol{S}^\top \boldsymbol{S}\boldsymbol{V}^\top,$$

so the eigenvalues of $\boldsymbol{A}^\top \boldsymbol{A}$ are σ_i^2. (And similarly, eigenvalues of $\boldsymbol{A}\boldsymbol{A}^\top$ are also σ_i^2.)

Key idea: Norms through singular values

$$\|\boldsymbol{A}\|_{\mathrm{op}} = \sigma_{\max}(\boldsymbol{A}) = \sigma_1, \qquad \|\boldsymbol{A}\|_F^2 = \mathrm{tr}(\boldsymbol{A}^\top \boldsymbol{A}) = \sum_i \sigma_i^2.$$

If $\boldsymbol{A}$ is square, then

$$|\det(\boldsymbol{A})| = \prod_i \sigma_i.$$

So singular values control both stretching (operator norm) and volume (determinant magnitude).

Definition: Nuclear norm

$$\|\boldsymbol{A}\|_* = \sum_i \sigma_i(\boldsymbol{A}).$$

(Used as a convex proxy for rank.)

Key idea: Norm relationships

If rank is r:

$$\|\boldsymbol{A}\|_{\mathrm{op}} \le \|\boldsymbol{A}\|_F \le \sqrt{r}\, \|\boldsymbol{A}\|_{\mathrm{op}}.$$

Definition: PCA (principal component analysis)

Center data and compute top principal directions via SVD (or covariance eigenvectors). Projecting onto them reduces dimensionality while preserving variance.

2.11 A few advanced-but-useful identities (optional)

Key idea: Matrix determinant lemma (rank-1 update)

If $\boldsymbol{A}$ is invertible and $\boldsymbol{u}, \boldsymbol{v}$ are vectors, then

$$\det(\boldsymbol{A} + \boldsymbol{u}\boldsymbol{v}^\top) = \det(\boldsymbol{A}) \left(1 + \boldsymbol{v}^\top \boldsymbol{A}^{-1} \boldsymbol{u}\right).$$

This is useful in Gaussian updates and low-rank approximations.

Key idea: Woodbury identity (inverse of low-rank update)

If $\boldsymbol{A}$ is invertible and $\boldsymbol{C}$ is invertible,

$$(\boldsymbol{A} + \boldsymbol{U}\boldsymbol{C}\boldsymbol{V})^{-1} = \boldsymbol{A}^{-1} - \boldsymbol{A}^{-1}\boldsymbol{U}\left(\boldsymbol{C}^{-1} + \boldsymbol{V}\boldsymbol{A}^{-1}\boldsymbol{U}\right)^{-1} \boldsymbol{V}\boldsymbol{A}^{-1}.$$

This explains why some "kernel" and "Gaussian process" computations can be made efficient.

2.12 Conditioning (why GD can zig-zag)

Definition: Condition number

For invertible $\boldsymbol{A}$ in 2-norm:

$$\kappa(\boldsymbol{A}) = \frac{\sigma_{\max}(\boldsymbol{A})}{\sigma_{\min}(\boldsymbol{A})}.$$

Key idea: Conditioning and optimization

Large κ means gradients point in directions with very different curvature scales. Gradient descent then takes "zig-zag" steps and converges slowly. Normalization and regularization often reduce effective conditioning issues.

2.13 Matrix calculus

Definition: Gradient w.r.t. a vector

For scalar $f(\boldsymbol{x})$, $\nabla_{\boldsymbol{x}} f \in \mathbb{R}^d$ stacks partial derivatives.

Definition: Gradient w.r.t. a matrix

For scalar $f(\boldsymbol{X})$, $\nabla_{\boldsymbol{X}} f$ is the matrix of partial derivatives (same shape as $\boldsymbol{X}$).

Definition: Jacobian and Hessian

For $\boldsymbol{y} = g(\boldsymbol{x}) \in \mathbb{R}^m$, Jacobian $J_g(\boldsymbol{x}) \in \mathbb{R}^{m \times d}$ has $(J_g)_{ij} = \partial y_i / \partial x_j$.
For scalar $f(\boldsymbol{x})$, Hessian $\nabla^2 f(\boldsymbol{x}) \in \mathbb{R}^{d \times d}$.

Key idea: Chain rule (backprop form)

If $\boldsymbol{y} = g(\boldsymbol{x})$ and $f = h(\boldsymbol{y})$ (scalar), then

$$\nabla_{\boldsymbol{x}} f = J_g(\boldsymbol{x})^\top \nabla_{\boldsymbol{y}} f.$$

Key idea: Differential form (shape-safe derivations)

For scalar $f(\boldsymbol{X})$:

$$df = \operatorname{tr}\big((\nabla_{\boldsymbol{X}} f)^\top d\boldsymbol{X}\big)\,.$$

This identity is the fastest route to correct matrix derivatives.

2.13.1 Differential rules (very useful in practice)

Key idea: Core differential identities

Assume matrices are compatible and constants have zero differential.

$$d(\boldsymbol{A}\boldsymbol{X}\boldsymbol{B}) = \boldsymbol{A}\,(d\boldsymbol{X})\,\boldsymbol{B}, \qquad d(\boldsymbol{X}^\top) = (d\boldsymbol{X})^\top.$$

$$d(\operatorname{tr}(\boldsymbol{X})) = \operatorname{tr}(d\boldsymbol{X}), \qquad d(\boldsymbol{X}^{-1}) = -\boldsymbol{X}^{-1}(d\boldsymbol{X})\boldsymbol{X}^{-1}.$$

$$d(\log\det \boldsymbol{X}) = \operatorname{tr}(\boldsymbol{X}^{-1} d\boldsymbol{X}) \quad (\boldsymbol{X} \text{ invertible}),$$

$$d(\det \boldsymbol{X}) = \det(\boldsymbol{X})\operatorname{tr}(\boldsymbol{X}^{-1} d\boldsymbol{X}) \quad (\boldsymbol{X} \text{ invertible}).$$

2.13.2 Scalar-from-vector gradients

Let $\boldsymbol{x}, \boldsymbol{a}, \boldsymbol{b} \in \mathbb{R}^d$, $\boldsymbol{A} \in \mathbb{R}^{d \times d}$, $\boldsymbol{X} \in \mathbb{R}^{n \times d}$, $\boldsymbol{y} \in \mathbb{R}^n$.

$f(\boldsymbol{x})$	$\nabla_{\boldsymbol{x}} f$
c	$\mathbf{0}$
$\boldsymbol{a}^\top \boldsymbol{x}$	$\boldsymbol{a}$
$\boldsymbol{x}^\top \boldsymbol{y}$ (treat $\boldsymbol{y}$ constant)	$\boldsymbol{y}$
$\frac{1}{2}\|\boldsymbol{x}\|_2^2$	$\boldsymbol{x}$
$\|\boldsymbol{x}\|_2^2$	$2\boldsymbol{x}$
$\|\boldsymbol{x}-\boldsymbol{a}\|_2^2$	$2(\boldsymbol{x}-\boldsymbol{a})$
$\boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x}$	$(\boldsymbol{A}+\boldsymbol{A}^\top)\boldsymbol{x}$
$\frac{1}{2}\|\boldsymbol{X}\boldsymbol{x}-\boldsymbol{y}\|_2^2$	$\boldsymbol{X}^\top(\boldsymbol{X}\boldsymbol{x}-\boldsymbol{y})$
$\log(\boldsymbol{a}^\top \boldsymbol{x})$ (domain > 0)	$\frac{\boldsymbol{a}}{\boldsymbol{a}^\top \boldsymbol{x}}$
$\mathrm{LSE}(\boldsymbol{x}) = \log \sum_i e^{x_i}$	$\mathrm{softmax}(\boldsymbol{x})$

Key idea: A very common quadratic form

If $\boldsymbol{\Sigma} \succ 0$ is symmetric and

$$f(\boldsymbol{x}) = \frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}),$$

then

$$\nabla_{\boldsymbol{x}} f = \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}).$$

This appears in Gaussian NLLs and Mahalanobis distances.

Common pitfall: Non-differentiable points

Functions like $\|\boldsymbol{x}\|_1$ and ReLU are not differentiable at some points. In optimization we use **subgradients** (valid generalized gradients) there.

2.13.3 Scalar-from-matrix gradients

Let $\boldsymbol{X} \in \mathbb{R}^{m\times n}$, constants $\boldsymbol{A}, \boldsymbol{B}$ are compatible.

$f(\boldsymbol{X})$	$\nabla_{\boldsymbol{X}} f$
$\mathrm{tr}(\boldsymbol{A}^\top \boldsymbol{X})$	$\boldsymbol{A}$
$\mathrm{tr}(\boldsymbol{A}\boldsymbol{X}\boldsymbol{B})$	$\boldsymbol{A}^\top \boldsymbol{B}^\top$
$\frac{1}{2}\|\boldsymbol{X}\|_F^2 = \frac{1}{2}\mathrm{tr}(\boldsymbol{X}^\top \boldsymbol{X})$	$\boldsymbol{X}$
$\|\boldsymbol{X}\|_F^2 = \mathrm{tr}(\boldsymbol{X}^\top \boldsymbol{X})$	$2\boldsymbol{X}$
$\frac{1}{2}\|\boldsymbol{A}\boldsymbol{X}-\boldsymbol{B}\|_F^2$	$\boldsymbol{A}^\top(\boldsymbol{A}\boldsymbol{X}-\boldsymbol{B})$
$\frac{1}{2}\|\boldsymbol{X}\boldsymbol{A}-\boldsymbol{B}\|_F^2$	$(\boldsymbol{X}\boldsymbol{A}-\boldsymbol{B})\boldsymbol{A}^\top$
$\mathrm{tr}(\boldsymbol{X}^\top \boldsymbol{A}\boldsymbol{X})$	$(\boldsymbol{A}+\boldsymbol{A}^\top)\boldsymbol{X}$
$\log\det(\boldsymbol{X})$ (invertible)	$(\boldsymbol{X}^{-1})^\top$
$-\log\det(\boldsymbol{X})$ (invertible)	$-(\boldsymbol{X}^{-1})^\top$
$\det(\boldsymbol{X})$ (invertible)	$\det(\boldsymbol{X})\,(\boldsymbol{X}^{-1})^\top$
$\mathrm{tr}(\boldsymbol{X}^{-1}\boldsymbol{A})$ (invertible)	$-(\boldsymbol{X}^{-1}\boldsymbol{A}\boldsymbol{X}^{-1})^\top$

2.13.4 Jacobians you use constantly (NN building blocks)

Definition: Elementwise nonlinearity

If $\boldsymbol{y} = \phi(\boldsymbol{x})$ elementwise (e.g., ReLU, tanh, sigmoid), then $J_\phi(\boldsymbol{x})$ is diagonal with entries $\phi'(x_i)$.

Definition: ReLU and sigmoid

$\mathrm{ReLU}(t) = \max(0, t)$ with derivative $\mathrm{ReLU}'(t) = \mathbb{1}\{t > 0\}$ (choose a subgradient at $t = 0$).
$\sigma(t) = \frac{1}{1+e^{-t}}$ with $\sigma'(t) = \sigma(t)(1 - \sigma(t))$.

Definition: Softmax Jacobian

If $\boldsymbol{p} = \mathrm{softmax}(\boldsymbol{z})$, then

$$\frac{\partial p_i}{\partial z_j} = p_i(\delta_{ij} - p_j).$$

In matrix form: $J_{\mathrm{softmax}}(\boldsymbol{z}) = \mathrm{diag}(\boldsymbol{p}) - \boldsymbol{p}\boldsymbol{p}^\top$.

Key idea: Log-sum-exp and softmax

$$\mathrm{LSE}(\boldsymbol{z}) = \log \sum_k e^{z_k} \quad \Rightarrow \quad \nabla_{\boldsymbol{z}} \mathrm{LSE}(\boldsymbol{z}) = \mathrm{softmax}(\boldsymbol{z}).$$

This is the backbone of stable softmax computations.

Key idea: Softmax + cross-entropy gradient shortcut

With one-hot $\boldsymbol{y}$, $\ell(\boldsymbol{z}) = -\sum_k y_k \log p_k$ and $\boldsymbol{p} = \mathrm{softmax}(\boldsymbol{z})$:

$$\nabla_{\boldsymbol{z}} \ell = \boldsymbol{p} - \boldsymbol{y}.$$

Equivalently, if the "target distribution" is $\boldsymbol{q}$ (not necessarily one-hot), then $\nabla_{\boldsymbol{z}} H(\boldsymbol{q}, \boldsymbol{p}) = \boldsymbol{p} - \boldsymbol{q}$.

2.13.5 Gradients for affine layers

Definition: Affine layer

Let $\boldsymbol{x} \in \mathbb{R}^d$, $\boldsymbol{W} \in \mathbb{R}^{m \times d}$, $\boldsymbol{b} \in \mathbb{R}^m$, and

$$\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}.$$

Key idea: Backprop through an affine layer

Let $\boldsymbol{g} = \frac{\partial \ell}{\partial \boldsymbol{y}} \in \mathbb{R}^m$ be the "upstream gradient". Then:

$$\frac{\partial \ell}{\partial \boldsymbol{W}} = \boldsymbol{g}\boldsymbol{x}^\top, \qquad \frac{\partial \ell}{\partial \boldsymbol{b}} = \boldsymbol{g}, \qquad \frac{\partial \ell}{\partial \boldsymbol{x}} = \boldsymbol{W}^\top \boldsymbol{g}.$$

These three lines are responsible for most manual NN gradient checks.

2.13.6 Subgradients (optional)

Definition: Subgradient (informal)

For convex f, a vector $\boldsymbol{g}$ is a subgradient at $\boldsymbol{x}$ if $f(\boldsymbol{z}) \geq f(\boldsymbol{x}) + \boldsymbol{g}^\top(\boldsymbol{z} - \boldsymbol{x})$ for all $\boldsymbol{z}$. At differentiable points, the subgradient equals the gradient.

Key idea: Common subgradients

For $|t|$:

$$\partial|t| = \begin{cases} \{1\}, & t > 0 \\ [-1, 1], & t = 0 \\ \{-1\}, & t < 0 \end{cases}$$

For $\|\boldsymbol{x}\|_1 = \sum_i |x_i|$, subgradients apply elementwise.
For ReLU at 0, a common choice is 0.

3 Part III — Optimization

3.1 Optimization problems in ML

Definition: Optimization objective

An optimization problem chooses parameters θ to minimize a scalar objective:

$$\min_\theta f(\theta).$$

In ML, f is usually "average loss (empirical risk) + regularization".

Definition: Empirical risk minimization (ERM)

Given data (x_i, y_i) and loss $\ell(\theta; x_i, y_i)$:

$$\min_\theta \ \frac{1}{n} \sum_{i=1}^{n} \ell(\theta; x_i, y_i).$$

Definition: Regularization

Add $\lambda\Omega(\theta)$:

$$\min_\theta \ \frac{1}{n} \sum_{i=1}^{n} \ell(\theta; x_i, y_i) + \lambda\Omega(\theta).$$

Often $\Omega(\theta) = \frac{1}{2}\|\theta\|_2^2$ (weight decay) or $\Omega(\theta) = \|\theta\|_1$ (sparsity).

3.2 First-order methods: gradient descent and SGD

Definition: Gradient

For differentiable $f(\theta)$, $\nabla f(\theta)$ points in steepest ascent direction. To minimize, move in the negative gradient direction.

Definition: Gradient descent (GD)

$$\theta_{t+1} = \theta_t - \eta \nabla f(\theta_t),$$

with learning rate $\eta > 0$.

Definition: Stochastic gradient descent (SGD)

If $f(\theta) = \frac{1}{n}\sum_{i=1}^{n} f_i(\theta)$, SGD uses a mini-batch B:

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{1}{|B|} \sum_{i \in B} \nabla f_i(\theta_t).$$

Key idea: Why SGD works

The mini-batch gradient is an unbiased (or approximately unbiased) estimate of the full gradient. Its noise can help exploration and sometimes improves generalization.

3.3 Step sizes, smoothness, and why learning rate is critical

Definition: Learning rate

The learning rate η controls the update magnitude. Too large: divergence/oscillation. Too small: very slow convergence.

Definition: L-smoothness (Lipschitz gradient, concept)

f is L-smooth if $\|\nabla f(x) - \nabla f(y)\|_2 \leq L \|x - y\|_2$. Heuristically, $1/L$ is a safe step-size scale.

Key idea: Conditioning and step sizes

If curvature differs strongly across directions (ill-conditioned), a single learning rate may be too big for one direction and too small for another. This motivates momentum, adaptive methods, and normalization.

3.4 Momentum and Adam

Definition: Momentum

Maintain velocity $\boldsymbol{v}$:

$$\boldsymbol{v}_{t+1} = \beta \boldsymbol{v}_t + (1-\beta)\nabla f(\theta_t), \qquad \theta_{t+1} = \theta_t - \eta \boldsymbol{v}_{t+1}.$$

Definition: Adam (high-level)

Adam maintains moving averages of gradients and squared gradients to adapt step sizes per parameter. You typically use a library implementation; the important concept is adaptive scaling + momentum.

Definition: Learning-rate schedules

A schedule changes η over time (warmup, step decay, cosine decay). Warmup often stabilizes early training; decay often improves final convergence.

3.5 Second-order intuition

Definition: Hessian

For scalar $f(\boldsymbol{x})$, the Hessian $\nabla^2 f(\boldsymbol{x})$ captures curvature.

Key idea: Newton step (idea)

Newton's method updates using curvature:

$$\boldsymbol{x}_{t+1} = \boldsymbol{x}_t - (\nabla^2 f(\boldsymbol{x}_t))^{-1} \nabla f(\boldsymbol{x}_t).$$

In deep learning this is typically too expensive, but it explains why curvature-aware updates can be faster.

3.6 Convexity and constraints

Definition: Convex function

f is convex if for all x, y and $\alpha \in [0,1]$:

$$f(\alpha x + (1-\alpha)y) \leq \alpha f(x) + (1-\alpha) f(y).$$

Key idea: Why convexity matters

In convex problems, any local minimum is a global minimum. Deep nets are nonconvex, but many subproblems (e.g., least squares, logistic regression) are convex.

Definition: Constraints

A constrained problem has $g(x) \leq 0$ or $h(x) = 0$. ML examples: probabilities sum to 1, norm constraints, simplex constraints.

Key idea: How ML handles constraints in practice

Common methods: parameterization (softmax for simplex), projections, penalty terms, or Lagrangians.

3.7 Lagrange multipliers, KKT conditions, and constrained optimization

Definition: Constrained optimization (standard form)

A common constrained problem is

$$\min_{\boldsymbol{x} \in \mathbb{R}^d} f(\boldsymbol{x}) \quad \text{s.t.} \quad g_i(\boldsymbol{x}) \leq 0 \ (i = 1, \ldots, m), \qquad h_j(\boldsymbol{x}) = 0 \ (j = 1, \ldots, p).$$

Definition: Lagrangian

The **Lagrangian** combines the objective and constraints via multipliers:

$$\mathcal{L}(\boldsymbol{x}, \boldsymbol{\lambda}, \boldsymbol{\nu}) = f(\boldsymbol{x}) + \sum_{i=1}^{m} \lambda_i \, g_i(\boldsymbol{x}) + \sum_{j=1}^{p} \nu_j \, h_j(\boldsymbol{x}),$$

where $\boldsymbol{\lambda} \in \mathbb{R}^m$ are multipliers for inequality constraints and $\boldsymbol{\nu} \in \mathbb{R}^p$ are multipliers for equality constraints.

Key idea: Why Lagrangians matter in ML

Lagrangians are the conceptual foundation behind:

- **Soft constraints / penalties** (e.g., adding $\lambda \|w\|^2$ as a regularizer).
- **Projection methods** (e.g., projected gradient descent).
- **Duality** (e.g., SVM dual, some kernel methods, convex optimization tooling).

Even when you do not explicitly solve dual problems, KKT conditions explain what "optimal under constraints" means.

3.7.1 Lagrange multipliers (equality constraints)

Definition: Lagrange multiplier rule (equality constraints)

For

$$\min_{\boldsymbol{x}} f(\boldsymbol{x}) \quad \text{s.t.} \quad h(\boldsymbol{x}) = 0,$$

a necessary condition (under mild regularity assumptions) is that there exists ν such that

$$\nabla f(\boldsymbol{x}^\star) + \nu \, \nabla h(\boldsymbol{x}^\star) = \mathbf{0}, \qquad h(\boldsymbol{x}^\star) = 0.$$

Geometric meaning: at the optimum, the objective gradient is orthogonal to the feasible surface.

Example: Minimize distance to a line

Solve $\min_{\boldsymbol{x}} \ \frac{1}{2}\|\boldsymbol{x}-\boldsymbol{a}\|_2^2$ subject to $\boldsymbol{c}^\top \boldsymbol{x} = d$. Lagrangian:

$$\mathcal{L}(\boldsymbol{x},\nu) = \tfrac{1}{2}\|\boldsymbol{x}-\boldsymbol{a}\|_2^2 + \nu(\boldsymbol{c}^\top \boldsymbol{x} - d).$$

Stationarity gives $\boldsymbol{x} - \boldsymbol{a} + \nu \boldsymbol{c} = \boldsymbol{0}$, hence $\boldsymbol{x} = \boldsymbol{a} - \nu \boldsymbol{c}$. Enforce the constraint:

$$\boldsymbol{c}^\top(\boldsymbol{a} - \nu \boldsymbol{c}) = d \ \Rightarrow \ \nu = \frac{\boldsymbol{c}^\top \boldsymbol{a} - d}{\|\boldsymbol{c}\|_2^2}.$$

So $\boldsymbol{x}^\star$ is the Euclidean projection of $\boldsymbol{a}$ onto the hyperplane.

3.7.2 KKT conditions (inequalities + equalities; optional)

Definition: KKT conditions

Consider

$$\min_{\boldsymbol{x}} f(\boldsymbol{x}) \quad \text{s.t.} \quad g_i(\boldsymbol{x}) \le 0, \ i = 1,\dots,m, \qquad h_j(\boldsymbol{x}) = 0, \ j = 1,\dots,p.$$

A point $\boldsymbol{x}^\star$ satisfies the **Karush–Kuhn–Tucker (KKT) conditions** if there exist multipliers $\boldsymbol{\lambda}^\star \in \mathbb{R}^m$, $\boldsymbol{\nu}^\star \in \mathbb{R}^p$ such that:

- **Primal feasibility:**
$$g_i(\boldsymbol{x}^\star) \le 0 \ \forall i, \qquad h_j(\boldsymbol{x}^\star) = 0 \ \forall j.$$
- **Dual feasibility:**
$$\lambda_i^\star \ge 0 \ \forall i.$$
- **Stationarity:**
$$\nabla_{\boldsymbol{x}} \mathcal{L}(\boldsymbol{x}^\star, \boldsymbol{\lambda}^\star, \boldsymbol{\nu}^\star) = \nabla f(\boldsymbol{x}^\star) + \sum_{i=1}^m \lambda_i^\star \nabla g_i(\boldsymbol{x}^\star) + \sum_{j=1}^p \nu_j^\star \nabla h_j(\boldsymbol{x}^\star) = \boldsymbol{0}.$$
- **Complementary slackness:**
$$\lambda_i^\star \, g_i(\boldsymbol{x}^\star) = 0 \quad \forall i.$$

For convex problems (convex f, convex g_i, affine h_j) plus a mild constraint qualification (e.g., Slater's condition), KKT conditions are not only necessary but also sufficient for optimality.

Key idea: Interpretation of complementary slackness

For each inequality constraint $g_i(\boldsymbol{x}) \le 0$:

- If the constraint is **inactive** at optimum (strictly satisfied: $g_i(\boldsymbol{x}^\star) < 0$), then $\lambda_i^\star = 0$ (it has no "force").
- If the constraint is **active** at optimum ($g_i(\boldsymbol{x}^\star) = 0$), then it may have $\lambda_i^\star > 0$ (it can affect the solution).

This is exactly the "active set" intuition used in constrained optimization.

Example: Projection onto an ℓ_2 ball (classic KKT toy example)

Problem:

$$\min_{\boldsymbol{x}} \tfrac{1}{2}\|\boldsymbol{x}-\boldsymbol{a}\|_2^2 \quad \text{s.t.} \quad \|\boldsymbol{x}\|_2 \le r.$$

Write constraint as $g(\boldsymbol{x}) = \|\boldsymbol{x}\|_2^2 - r^2 \le 0$ and Lagrangian

$$\mathcal{L}(\boldsymbol{x},\lambda) = \tfrac{1}{2}\|\boldsymbol{x}-\boldsymbol{a}\|_2^2 + \lambda(\|\boldsymbol{x}\|_2^2 - r^2), \quad \lambda \ge 0.$$

Stationarity:

$$(\boldsymbol{x}-\boldsymbol{a}) + 2\lambda\boldsymbol{x} = \boldsymbol{0} \;\Rightarrow\; \boldsymbol{x} = \frac{1}{1+2\lambda}\boldsymbol{a}.$$

If $\|\boldsymbol{a}\|_2 \le r$, optimum is $\boldsymbol{x}^\star = \boldsymbol{a}$ (constraint inactive, $\lambda^\star = 0$). If $\|\boldsymbol{a}\|_2 > r$, constraint active so $\|\boldsymbol{x}^\star\|_2 = r$, giving

$$\boldsymbol{x}^\star = r\frac{\boldsymbol{a}}{\|\boldsymbol{a}\|_2}.$$

So the solution is exactly "clip the vector onto the ball".

Recipe: Projected gradient descent (practical constrained optimization)

To solve $\min f(\boldsymbol{x})$ subject to $\boldsymbol{x} \in \mathcal{C}$ (a simple constraint set), iterate:

$$\boldsymbol{x}_{t+\frac{1}{2}} = \boldsymbol{x}_t - \eta\nabla f(\boldsymbol{x}_t), \qquad \boldsymbol{x}_{t+1} = \Pi_{\mathcal{C}}(\boldsymbol{x}_{t+\frac{1}{2}}),$$

where $\Pi_{\mathcal{C}}$ is the Euclidean projection onto $\mathcal{C}$. Examples: simplex constraints (probabilities), ℓ_2 balls, box constraints.

Common pitfall: KKT conditions need regularity assumptions

For nonconvex problems, KKT points may be saddle points or local minima. Also, KKT conditions may fail to characterize optima if constraint qualifications do not hold. In ML, we still use these conditions mainly as intuition and for convex subproblems.

3.8 Deep learning optimization issues and fixes

Definition: Vanishing gradients

Gradients become extremely small in deep networks, slowing learning in early layers.

Definition: Exploding gradients

Gradients become extremely large, causing numerical instability.

Key idea: Standard mitigations

Initialization, normalization (BatchNorm/LayerNorm), residual connections, gradient clipping, and careful learning-rate schedules.

Common pitfall: Silent numerical issues

If loss becomes NaN/Inf, check: data normalization, logits vs probabilities, mixed precision scaling, and extreme learning rates.

High-frequency cheat sheets

Probability / Statistics

- Bayes: $p(a \mid b) = \frac{p(b|a)p(a)}{p(b)}$.
- $\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$.
- $\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$.
- $\mathrm{Cov}(X, Y) = \mathbb{E}[XY] - \mathbb{E}[X]\mathbb{E}[Y]$.
- Total expectation: $\mathbb{E}[X] = \mathbb{E}(\mathbb{E}[X \mid Y])$.
- Total variance: $\mathrm{Var}(X) = \mathbb{E}[\mathrm{Var}(X \mid Y)] + \mathrm{Var}(\mathbb{E}[X \mid Y])$.

Information theory

- $H(p) = \mathbb{E}_p[-\log p(X)]$.
- $H(p, q) = \mathbb{E}_p[-\log q(X)]$.
- $\mathrm{D}_{\mathrm{KL}}(p\|q) = \mathbb{E}_p[\log p(X) - \log q(X)]$.
- $H(p, q) = H(p) + \mathrm{D}_{\mathrm{KL}}(p\|q)$.

Linear algebra

- Rank: $\mathrm{rank}(A) = \#$ independent columns.
- Orthogonal: $Q^\top Q = I$ (preserves lengths).
- $\mathrm{tr}(A) = \sum_i \lambda_i$; $\det(A) = \prod_i \lambda_i$.
- $\|A\|_F^2 = \mathrm{tr}(A^\top A) = \sum_i \sigma_i^2$.
- $\|A\|_{\mathrm{op}} = \sigma_{\max}(A)$.
- Condition number: $\kappa(A) = \sigma_{\max}/\sigma_{\min}$.

Derivatives (must-know)

- $\nabla_x(a^\top x) = a$.
- $\nabla_x \frac{1}{2}\|x\|_2^2 = x$.
- $\nabla_w \frac{1}{2}\|Xw - y\|_2^2 = X^\top(Xw - y)$.
- $\nabla_X \frac{1}{2}\|AX - B\|_F^2 = A^\top(AX - B)$.
- $\nabla_X \log\det(X) = (X^{-1})^\top$.
- $\nabla_z \mathrm{LSE}(z) = \mathrm{softmax}(z)$.
- Softmax-CE: $\nabla_z \ell = p - y$.

Optimization

- GD: $\theta \leftarrow \theta - \eta\nabla f(\theta)$.
- SGD: mini-batch estimate of gradient.
- Momentum: smooth gradients to reduce zig-zag.
- Adam: momentum + adaptive per-parameter scaling.